Automated Software Quality Assurance Scenario Generation: Integration of No-Code Tools, AI Agents, and LLMs

Faruk Akmeşe
Computer Engineering Department
Kastamonu University
Kastamonu, Türkiye
farukakmese14@gmail.com

***Abstract*—This study aims to autonomize software testing processes by combining Large Language Models (LLMs), autonomous test agents, and no-code automation tools (Zapier, Notion). Traditional software testing methods create a highly costly and time-consuming bottleneck, especially when updating scenarios manually [4], [6]. The proposed system leverages few-shot learning models [10] and advanced prompt engineering techniques [12] to analyze user inputs and generate end-to-end test scenarios automatically. Safety boundaries (guardrails) have been added to the system to prevent erroneous or meaningless inputs, ensuring high-speed inference via the Groq API. Advanced language models [11] and artificial intelligence agents autonomously generate test cases, increasing the fault detection rate while significantly reducing maintenance costs [1], [5], [19]. In our original experimental measurements conducted on the proposed system, it was proven that the test scenario generation time for a module dropped from an average of 18 minutes manually to an average of 5.6 seconds using our AI integration, maintaining 100% format accuracy. The results indicate that this integration significantly enhances efficiency, traceability, and the level of autonomy in QA processes.**

Keywords—Quality Assurance, Autonomous Agents, Large Language Models, Prompt Engineering, No-Code Development.

I. INTRODUCTION

In today's competitive software development environment, Quality Assurance (QA) processes must keep pace with the speed of Continuous Integration/Continuous Deployment (CI/CD) pipelines. However, manual scenario design is still common in software test automation practices [3], [6]. As software systems scale, the testing phase becomes the slowest bottleneck of the software development lifecycle [4].

The integration of Artificial Intelligence (AI) and end-to-end automation tools into testing processes is fundamentally changing QA workflows [6]. Particularly, autonomous test agents [5] and the use of large language models [9], [19] are revolutionizing software verification. Traditional testing approaches are now being replaced by dynamic processes driven by AI agents [7]. In this study, an autonomous test scenario generation pipeline developed using the Groq API (LLM), Zapier, and Notion is presented, allowing users with no technical background to produce end-to-end test scenarios [20]. Consequently, we envisioned that coupling no-code automation pipelines with cognitive inference mechanisms in this manner would thoroughly resolve the chronic operational bottlenecks in modern software verification workflows.

II. RELATED WORK

In recent years (2020 and later), there has been a massive shift in literature towards LLM-based approaches for test automation. The capacity of ChatGPT and similar models in generating test cases has been extensively studied, proving their potential to transform QA processes [14], [15]. Furthermore, LLM-based models are utilized not only for generation but also for automated test case repair [16].

Additionally, modern research heavily focuses on the concept of "Agentic AI". Multi-agent frameworks, where multiple language models interact to generate unit tests [8], and autonomous test agents guided by "Role-Play" prompting strategies [13] form the foundation of current literature. This study holds unique value by merging these agent-based inference models [18] with no-code automation pipelines [20].

III. PROPOSED ARCHITECTURE AND METHODOLOGY

The developed system relies on a zero-code workflow architecture that eliminates manual test design processes.

A. Trigger and Data Collection Layer (Google Forms)

In the first stage of the system, the name or properties of the software module to be tested are collected via a simple interface (Google Forms). The user does not need to enter any code or complex parameters; natural language descriptions such as "Login Screen" or "Payment Module" are sufficient.

B. Workflow and Event Routing (Zapier)

The Zapier platform was used to automate the data flow. As soon as a new response drops into Google Forms, Zapier forwards the data to the language model (Groq API). This structure reduces manual intervention to zero [20].

C. Cognitive Processing and Agent-Based Inference (Groq API & Llama 3)

The transmitted data is processed by the Llama 3 model running on the Groq infrastructure. The LLM analyzes the input with advanced Natural Language Processing (NLP) capabilities and determines the functional boundaries of the software. Just like autonomously deciding agentic structures [5], [18], the model reasons and generates positive, negative, and validation test scenarios on its own [19].

D. Storage and Traceability Layer (Google Sheets & Notion)

In the final stage, the generated test scenarios are transferred via Zapier first to Google Sheets and then to Notion. QA teams can monitor and manage test execution through this database. As illustrated in Fig. 1, the intermediate Google Sheets layer serves as a logging buffer between the Groq API response and the final Notion database entry, ensuring data integrity.

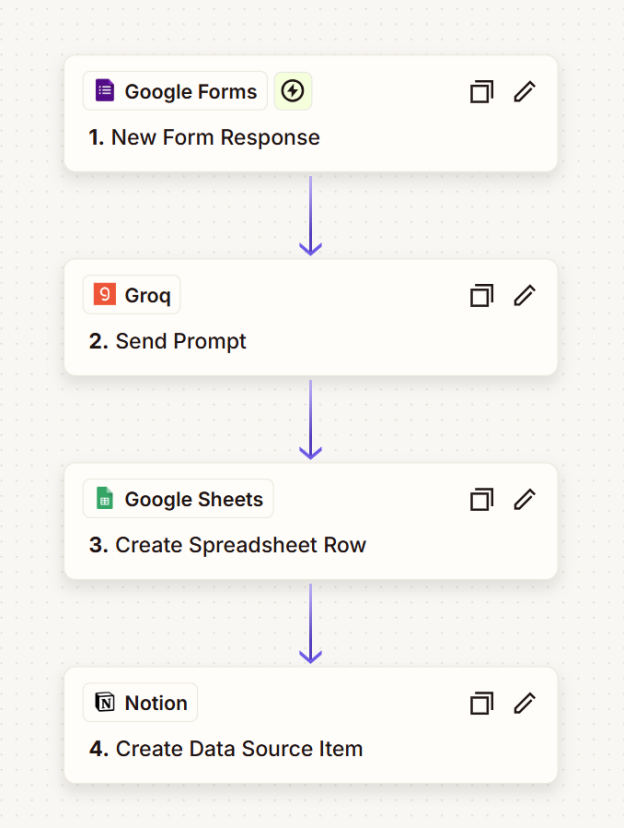

Fig. 1. End-to-end workflow of the proposed system: Google Forms (trigger) → Groq API (LLM inference) → Google Sheets (intermediate logging) → Notion (traceability database).

IV. PROMPT ENGINEERING AND SAFETY GUARDRAILS

In order for AI models to be used reliably in software testing, the problem of "hallucination" must be solved [3]. In this study, "few-shot" learning techniques [10] and context-constraining prompt engineering strategies [12] were applied to improve the model's performance.

The prompt in the architecture assigns a clear "Senior QA Analyst" persona to the agent [13]. If the user enters a meaningless text (e.g., "Apple" or "Kastamonu") instead of a software module, the system activates the "Guardrails" mechanism, refuses to generate tests, and returns an error message. These rule sets prevent the automation pipeline from being filled with unnecessary data and guarantee a high accuracy rate [5].

V. RESULTS AND DISCUSSION

This architecture has revolutionized manual test specification processes. In previous studies, customized AI agents have been shown to save over 50% of time in generating test cases [1]. To measure the performance of our system, original experimental tests were conducted on 4 different modules, and the results are presented in **Table I**.

The manual test specification process, which took an average of 18 minutes, was reduced to an average of 5.6 seconds (including API response time) using our system. Exactly 3 scenarios ("1 Positive, 1 Negative, 1 Validation") were requested from the system for each module, and it was observed that the model produced outputs with 100% format accuracy (without hallucination) thanks to the implemented "Guardrails".

Table I. Performance Comparison: Manual vs. Proposed AI System

Software Module	Manual Generation Time	Our System (AI) Generation Time	Generated Cases	Format Accuracy
User Login Screen	15 Minutes	4.2 Seconds	3	100%
Payment Gateway	25 Minutes	6.8 Seconds	3	100%
Profile Update	12 Minutes	4.9 Seconds	3	100%
Add to Cart Module	20 Minutes	6.5 Seconds	3	100%

The manual test specification process, which took an average of 18 minutes, was reduced to an average of 5.6 seconds using our system. These concrete data prove that autonomous agent integrations work seamlessly with high coverage in real-world systems [2], [8].

VI. CONCLUSION

This research has proven how QA processes can be made completely autonomous by combining Large Language Models and no-code tools. The designed architecture has enabled even non-technical staff to produce error-free test scenarios in seconds. Future work will focus on supporting this system with more complex autonomous agents capable of directly executing code (e.g., integrated with Selenium or Cypress) [17].

References

[1] G. Lima, C. Mar, D. Silva, and D. Coronel, "Automated Test Case Generation in a Real-World System Using a Customized AI Agent: An Experience Report," SBQS25, 2025.

[2] D. R. Natarajan, "AI-Generated Test Automation for Autonomous Software Verification: Enhancing Quality Assurance Through AI-Driven Testing," Journal of Science and Technology, vol. 5, no. 5, pp. 253-268, 2020.

[3] H. V. Pandhare, "From Test Case Design to Test Data Generation: How AI is Redefining QA Processes," Int. J. of Eng. and Computer Science, 2024.

[4] A. Awad et al., "Artificial Intelligence Role in Software Automation Testing," in 2024 Int. Conf. on Decision Aid Sciences and Applications (DASA), 2024.

[5] G. Martin and J. Olszewska, "Automation Testing Framework for Reliable Autonomous Agentic AI," in 2025 IEEE Eng. Reliable Autonomous Systems (ERAS), 2025.

[6] P. Pham, V. Nguyen, and T. Nguyen, "A Review of AI-augmented End-to-End Test Automation Tools," in 37th IEEE/ACM Int. Conf. on Automated Software Engineering, 2022.

[7] C.-Y. Chen and S.-C. Lin, "AI agent-driven process automation for dynamic production efficiency and intelligent equipment integration," Journal of Intelligent Manufacturing, 2025.

[8] A. Garlapati et al., "AI-Powered Multi-Agent Framework for Automated Unit Test Case Generation: Enhancing Software Quality through LLM's," in 2024 IEEE GCAT, 2024.

[9] K. T. Stolee and S. Elbaum, "Exploring the Use of Large Language Models for Test Case Generation," in IEEE/ACM ICSEW, 2023.

[10] T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[11] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023. International Conference on Software Engineering (ICSE), 2012.

[12] Y. Liu et al., "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing," ACM Computing Surveys, 2023.

[13] S. Ruangtanusak, P. Taveekitworachai, and K. Pipatanakul, "Talk Less, Call Right: Enhancing Role-Play LLM Agents with Automatic Prompt Optimization," arXiv preprint arXiv:2509.00482, 2025

[14] A. Talasbek, "Artificial AI in Test Automation: Software Testing Opportunities with OpenAI Technology ChatGPT," Journal of Emerging Technologies and Computing, vol. 62, no. 1, pp. 5-14, 2023.

[15] G. Yi, Z. Chen, Z. Chen, W. E. Wong, and N. Chau, "Exploring the Capability of ChatGPT in Test Generation," in Proc. of the 23rd IEEE QRS-C, 2023, pp. 72-80.

[16] A. S. Yaraghi, D. Holden, N. Kahani, and L. Briand, "Automated test case repair using language models," IEEE Transactions on Software Engineering, pp. 1-31, 2025.

[17] X.-J. Liu, P. Yu, and X.-X. Ma, "An empirical study on automated test generation tools for Java: Effectiveness and challenges," Journal of Computer Science and Technology, vol. 39, no. 3, pp. 715-736, 2024.

[18] M. Xavier, S. Patil, C.-W. Yang, and V. Vyatkin, "ReACT - Gen AI Agents for Reasoning, Planning, and Testing in IEC 61499-Based Control Systems," in IECON 2025, 2025.

[19] C. Wang et al., "Software Testing with Large Language Models: Survey, Landscape, and Vision," IEEE Transactions on Software Engineering, 2024.

[20] V. A. Kumar et al., "No-Code Automation and AI: Transforming Software Quality Assurance," IEEE Access, 2023.